

Web Client Report

Remote File Folder Client Project

(a simplified Google Drive analogue)

Yana Utochkina

Stage: Web (React/TypeScript) Client – Design, Usage, and
Testing
September 30, 2026

Abstract

This report documents the **React web client** of the **Remote File Folder Client** project (frontend/src/), a simplified analogue of Google Drive with web and desktop clients, a Go backend, PostgreSQL, and local-folder synchronization. It describes the client's internal structure, walks through its use cases with screenshots, and documents its unit test coverage, including its folder-sync feature: a browser-side port of the desktop client's sync engine, built on the File System Access API.

Contents

1	Introduction	2
2	Web Client Architecture	2
2.1	Internal Structure	2
2.2	Synchronization	3
3	Use Cases and Screenshots	3
3.1	Register and Authenticate	3
3.2	Browse Files: Attributes, Columns, Sorting, Filtering	4
3.3	View File Content	7
3.4	Upload, Download, Delete	8
3.5	Synchronize a Local Folder	11

4 Unit Testing **12**

4.1 Test Setup 12

4.2 Coverage by Area 13

5 Conclusions **14**

1. Introduction

The web client is one of two front ends (the other being the Swift desktop client) to a single Go API and PostgreSQL database, self-hosted on a home Ubuntu server behind a Cloudflare Tunnel. It covers every core workspace use case – registration, authentication, browsing files with sortable/filterable attributes, viewing `.java/.png` content, uploading (including drag-and-drop), downloading, deleting – and, in Chromium-based browsers, folder synchronization against the same server-side diff protocol the desktop client uses. Unlike the desktop client, a browser cannot watch a local folder continuously in the background, so sync here is a manual, one-shot action rather than an always-running service.

2. Web Client Architecture

2.1 Internal Structure

The web client’s source folder, `frontend/src/`, is organized as follows:

- **App.tsx**: the main `DriveView` component – `TanStack Query` for server state (a paginated `useInfiniteQuery` for the visible file list, a separate `useQuery` for the full extension list that feeds the filter dropdown), `TanStack Table` for sorting and column visibility, `TanStack Virtual` for windowed row rendering, and drag-and-drop upload handling;
- **columns.tsx**: the file table’s column definitions (name, created, modified, uploaded by, edited by, actions), including which extensions get a clickable preview link on their name cell;
- **auth/**: `AuthContext.tsx` (a `React Context` + hook holding the logged-in user and JWT access/refresh tokens, persisted to `localStorage`) and `LoginPage.tsx` (the combined login/register form);
- **api/**: `client.ts` (a fetch-based REST client with a one-refresh-one-retry policy on an expired access token, and a chunked-upload path for large files) and `types.ts` (the wire-format DTOs);
- **SettingsModal.tsx / FilePreviewModal.tsx**: column-visibility and file-content-preview modals, sharing the app’s generic modal chrome;
- **sync/**: the folder-sync feature – `hash.ts`, `folderScanner.ts`, `storage.ts`, `reconcile.ts` (the sync algorithm and state), and `SyncContext.tsx/SyncModal.tsx/SyncConflictsSection.tsx` (its `React state` and UI).

The web client talks to the Go API server over HTTPS at the same public Cloudflare Tunnel hostname the desktop client uses, and renders the same underlying file records the desktop client and the server both work with.

2.2 Synchronization

Folder synchronization mirrors the desktop client’s sync engine, adapted to what a browser can actually do: there is no background folder watcher, so a **Sync now** button drives a single reconcile pass on demand. Clicking it: scans the chosen folder via the File System Access API’s `FileSystemDirectoryHandle`, computes a manifest (name, size, SHA-256 per file, hashed with `crypto.subtle`), sends it together with the last-synced version to `POST /api/sync/diff` – the same endpoint and protocol the desktop client speaks – and applies the returned `download/delete_local/conflict` actions before uploading any new or changed local files and deleting the remote copies of any locally-removed ones. The sync cursor and per-file bookkeeping persist across page reloads in IndexedDB, alongside the chosen folder’s handle (which the browser can reuse without asking the user to reselect the folder every time, subject to a fresh permission grant per session).

A conflict action means the same file changed on both sides since the last sync. It is never resolved automatically: it is shown to the user with the local and remote size/editor, and **Keep Local** (re-upload, overwriting the remote copy), **Keep Remote** (overwrite the local file), or **Keep Both** (keep the local edit under a renamed copy and adopt the remote version) resolve it, exactly matching the desktop client’s own three-way resolution.

The File System Access API is available only in Chromium-based browsers (Chrome, Edge, Brave); on any other browser, the sync panel shows a plain message explaining this instead of attempting to use an API that isn’t there.

3. Use Cases and Screenshots

This section walks through the web client’s use cases, each with a screenshot of the running application.

3.1 Register and Authenticate

Register (fig. 1) and *Log in* (fig. 2) are two modes of the same form (`LoginPage.tsx`); a successful call to either stores the returned access/refresh JWT pair and the username in `localStorage` via `AuthContext`, after which every other view becomes reachable.

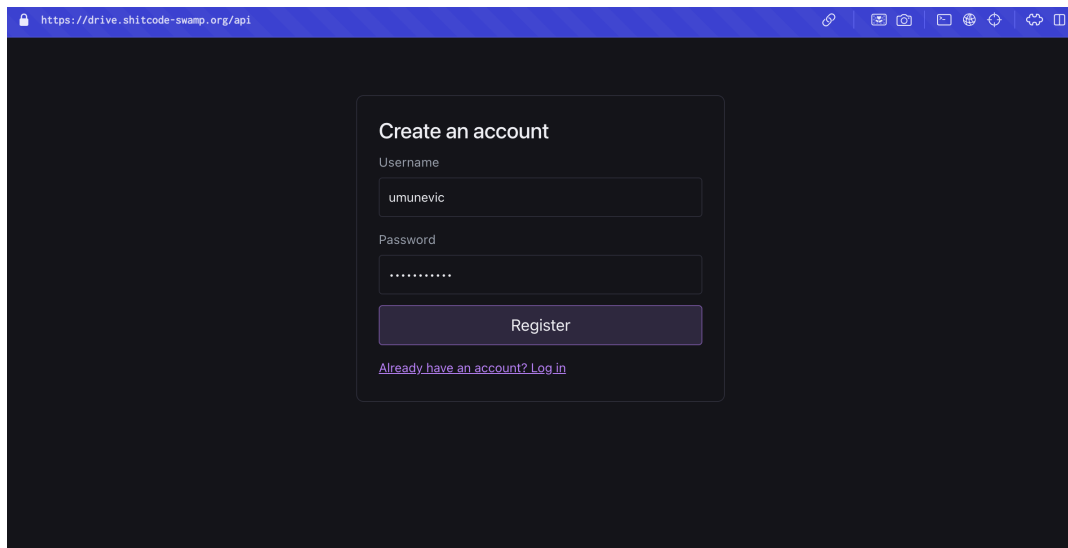


Figure 1: Registration screen

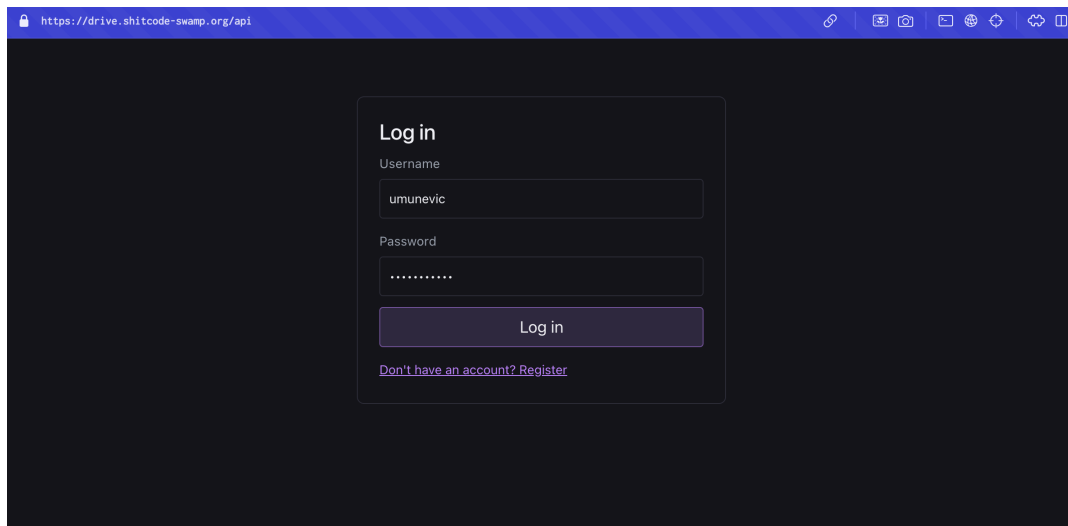


Figure 2: Login screen

3.2 Browse Files: Attributes, Columns, Sorting, Filtering

Figure 3 shows the main file table with every attribute column visible (name, created, modified, uploaded by, edited by); the name and actions columns cannot be hidden, while the others toggle through the Settings modal (fig. 4).

https://drive.shitcode-swamp.org/api

My Drive

Logged in as umunevic Folder Sync Settings Log out

Choose Files No file chosen Upload Filter: All files

Name ↑	Created ↑	Modified ↑	Uploaded by ↑	Edited by ↑	Actions
concrete-texture-black-wallpaper-background-photo.jpg	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete
Funny Cartoon Car Crash Sound Effect-(117k).mp3	30/09/2026, 17:15	30/09/2026, 17:15	umunevic	umunevic	Download Delete
kursova_mi_22_23_6_2_(1).pdf	30/09/2026, 15:44	30/09/2026, 15:44	umunevic	umunevic	Download Delete
pngtree-fruit-fresh-tomato-png-image_9959799.png	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete
Screenshot 2026-09-29 at 17.46.37.png	30/09/2026, 19:05	30/09/2026, 19:05	umunevic	umunevic	Download Delete
tg_image_4072793523.png	30/09/2026, 15:45	30/09/2026, 15:45	umunevic	umunevic	Download Delete
white-cloud-blue-sky-sea_74190-4488.avif	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete
БШНФОП18ПЛ19_ДВС_Прикладні_алг_2025_2026.doc	30/09/2026, 17:15	30/09/2026, 17:15	umunevic	umunevic	Download Delete
Лекція_3.docx	30/09/2026, 15:43	30/09/2026, 15:43	umunevic	umunevic	Download Delete
НП ШІ 2025-26-1.pdf	30/09/2026, 16:39	30/09/2026, 16:39	umunevic	umunevic	Download Delete

Figure 3: Main file list with all attribute columns visible

https://drive.shitcode-swamp.org/api

My Drive

Logged in as umunevic Folder Sync Settings Log out

Choose Files No file chosen Upload Filter: All files

Name ↑	Created ↑	Edited by ↑	Actions
concrete-texture-black-wallpaper-background-photo.jpg	30/09/2026, 16:39	umunevic	Download Delete
Funny Cartoon Car Crash Sound Effect-(117k).mp3	30/09/2026, 17:15	umunevic	Download Delete
kursova_mi_22_23_6_2_(1).pdf	30/09/2026, 15:44	umunevic	Download Delete
pngtree-fruit-f			Download Delete
Screenshot 20			Download Delete
tg_image_407			Download Delete
white-cloud-b			Download Delete
БШНФОП18П			Download Delete
Лекція_3.docx	30/09/2026, 15:43	umunevic	Download Delete
НП ШІ 2025-26-1.pdf	30/09/2026, 16:39	umunevic	Download Delete

Settings

Close

Columns

- ☒ Created
- ☐ Modified
- ☐ Uploaded by
- ☒ Edited by

Figure 4: Settings modal: toggling a non-name column

Sorting: clicking a column header sorts the table by that column, ascending or descending, shown by an arrow indicator next to the header (fig. 5 sorts by *Edited by*). Sorting and filtering are both applied server-side – each

change re-requests the current page from the API with the corresponding sort/order/extension query parameters, rather than re-ordering an already-fetched page in the browser.

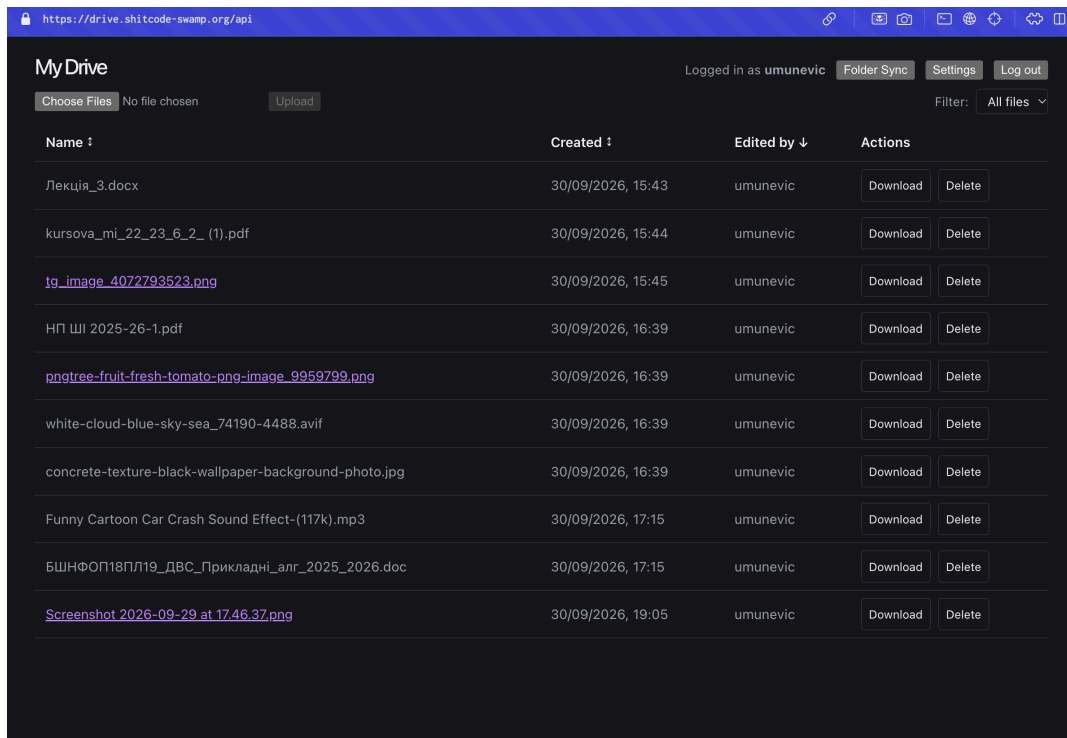


Figure 5: File list sorted by “Edited by”

Filtering: the extension dropdown, populated from every extension actually present among the user’s files (fetched independently of the current filter, so the option list never collapses to just the selected extension), filters the visible table down to just that extension or back to “All files” (fig. 6).

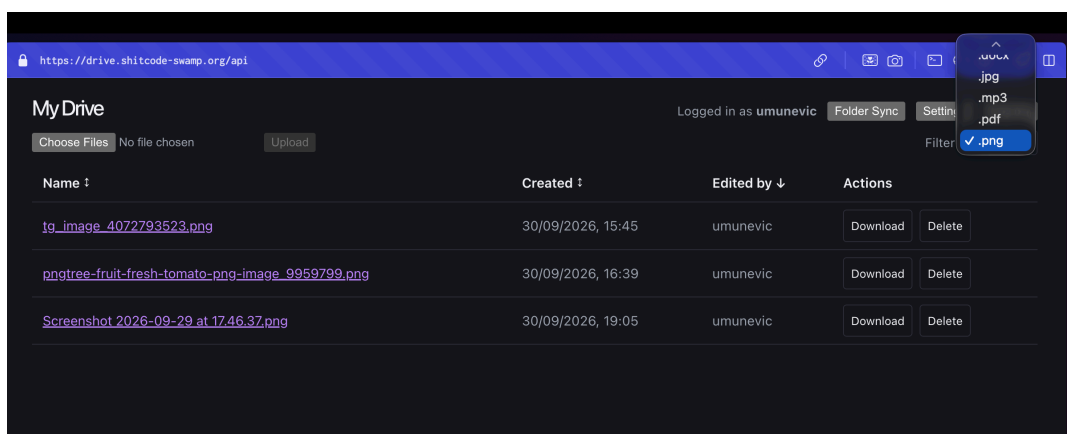


Figure 6: Extension filter applied

3.3 View File Content

Clicking a file's name previews its content if the extension is `.java` (rendered as text, fig. 7) or `.png` (rendered as an image, fig. 8); every other extension's name is plain, non-clickable text – it can still be downloaded, just not previewed inline.

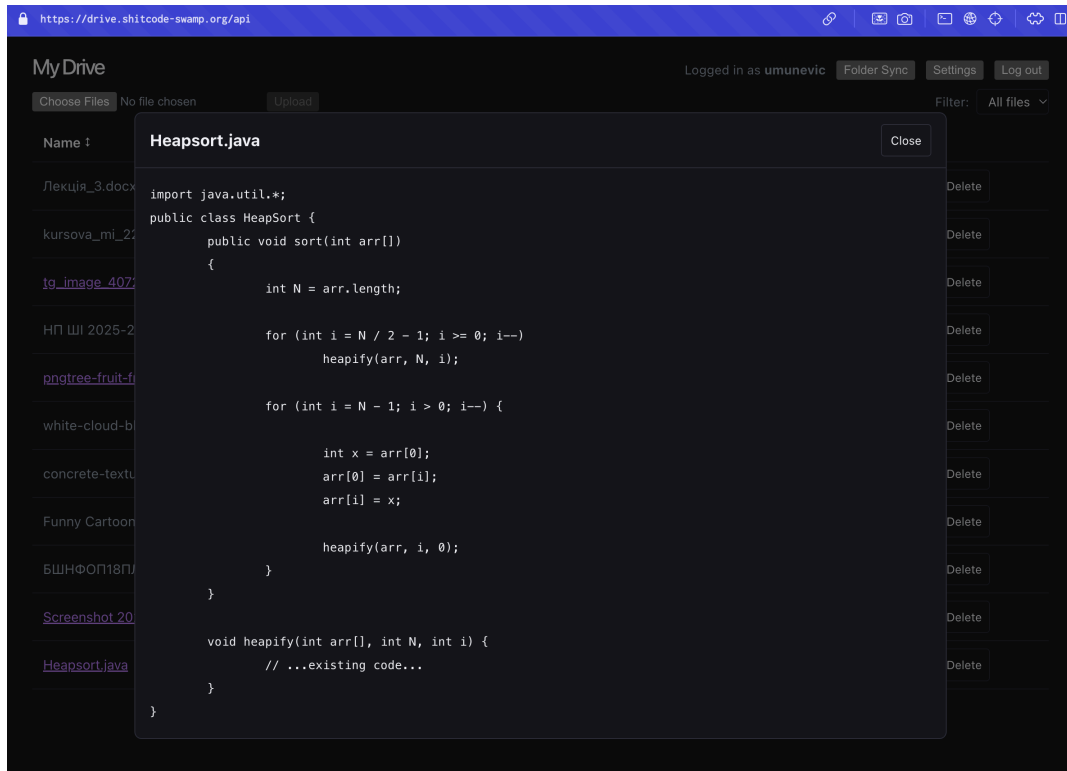


Figure 7: `.java` file previewed as text

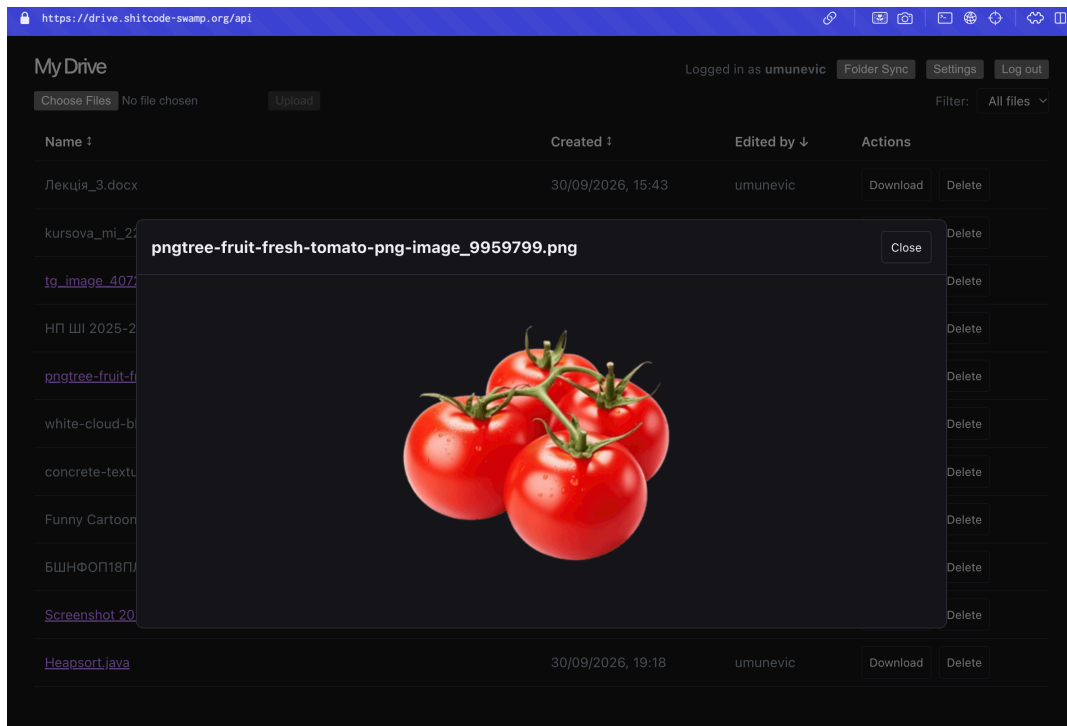


Figure 8: .png file previewed as an image

3.4 Upload, Download, Delete

Files can be uploaded either through the file picker and “Upload” button, or by dragging them anywhere onto the page – a full-page overlay appears while a drag carrying files is in progress (fig. 9). Figure 10 shows an upload in progress: large files are split into chunks so no single HTTP request exceeds the Cloudflare Tunnel free-tier body-size limit, with a progress bar per file reflecting bytes sent per chunk. Figures 11 and 12 show downloading a file and deleting one from the workspace.

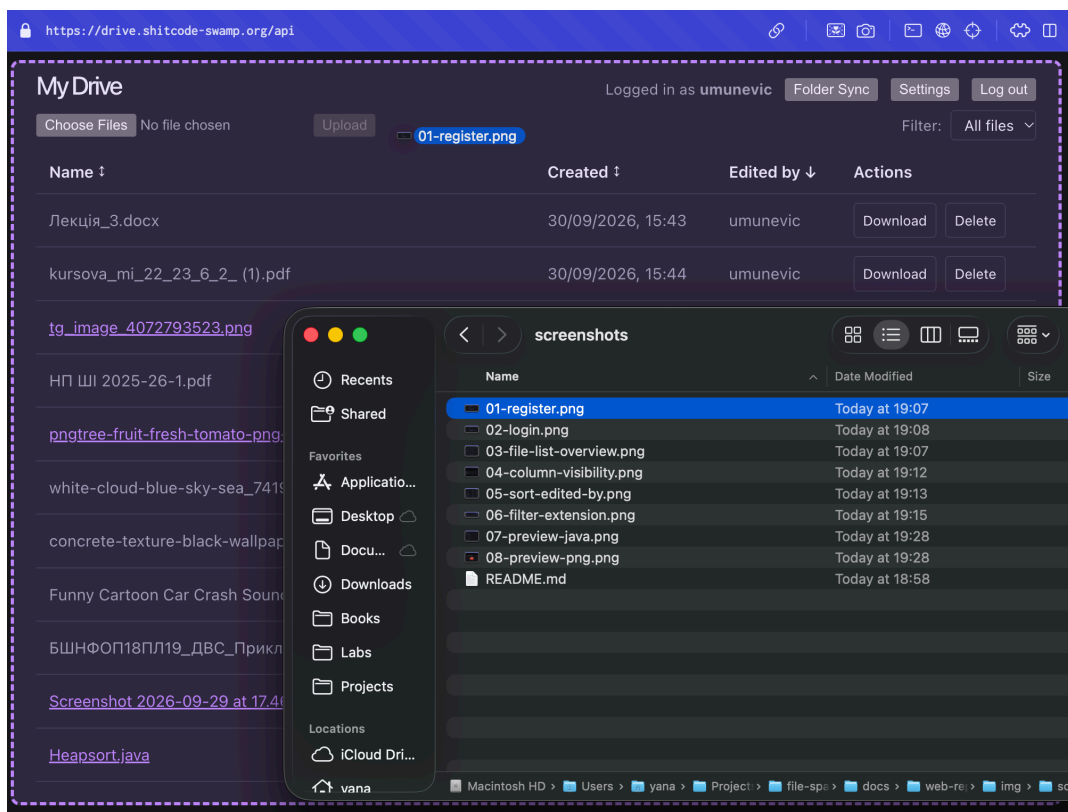


Figure 9: Drop overlay shown while dragging files onto the page

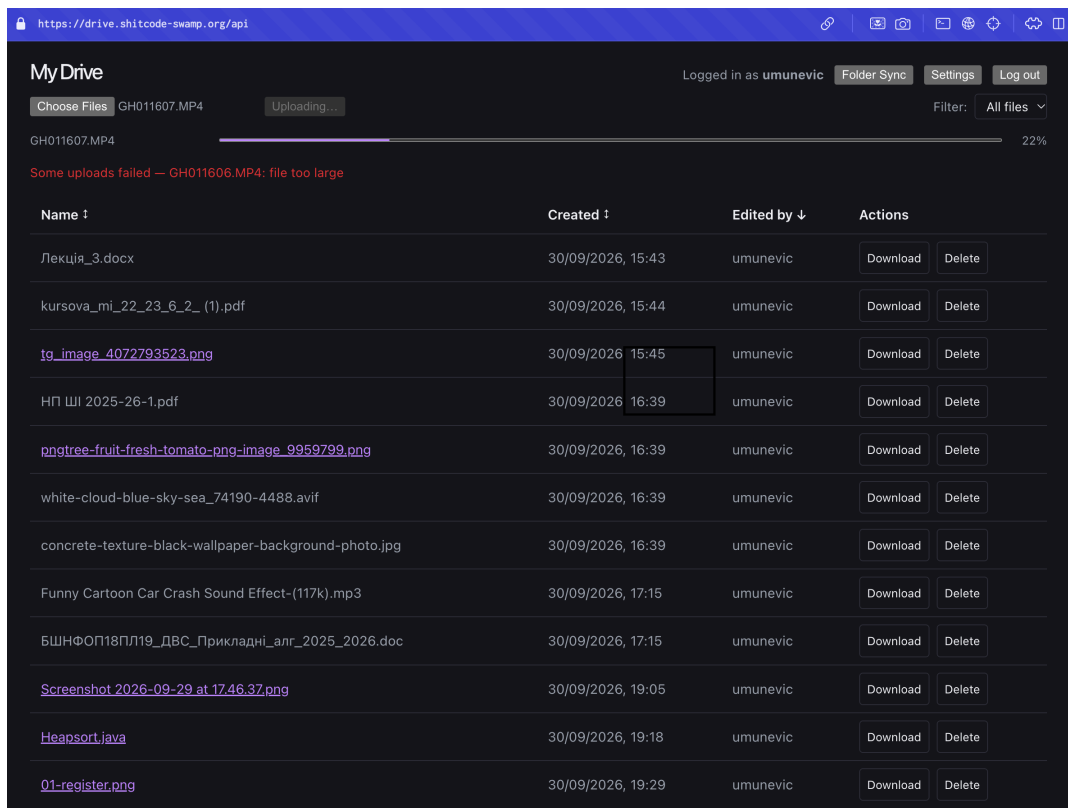


Figure 10: File upload in progress

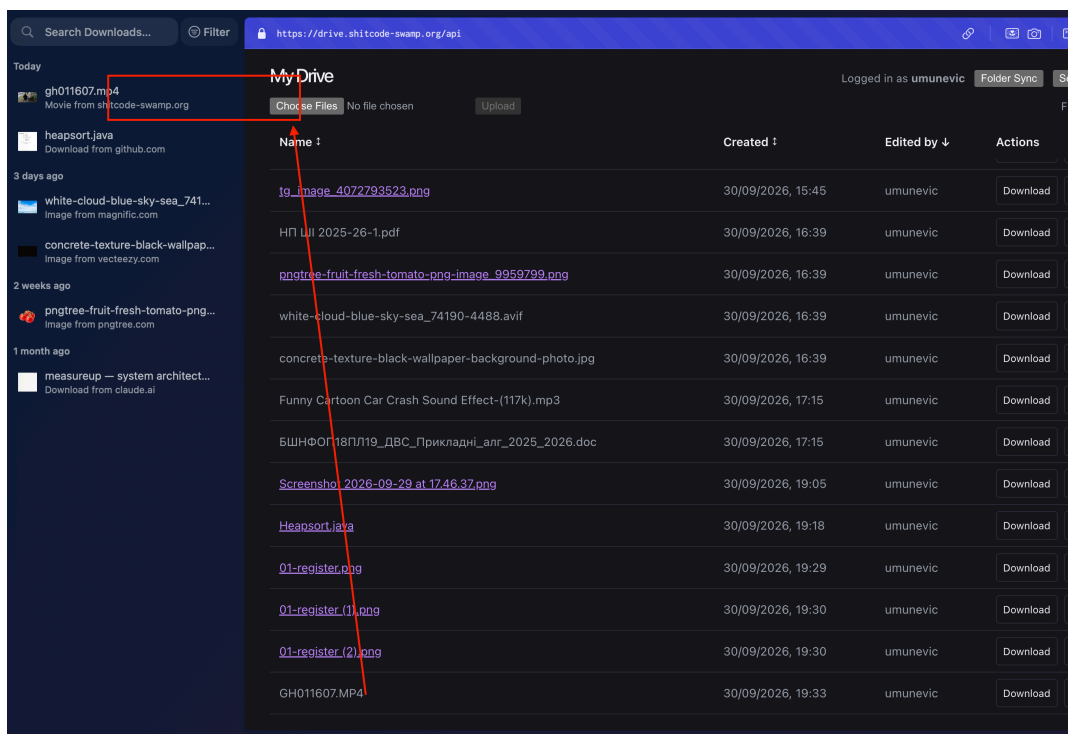


Figure 11: File download

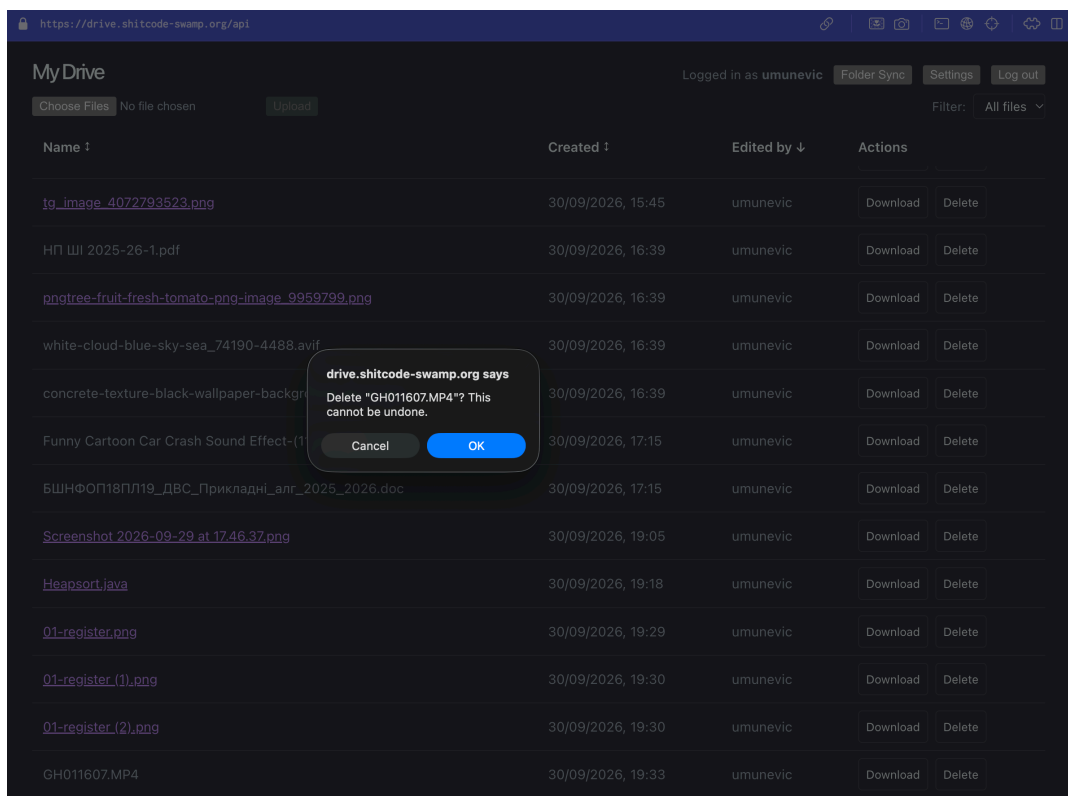


Figure 12: File deletion

3.5 Synchronize a Local Folder

Figure 13 shows the folder sync panel with a folder chosen and a sync just completed: the chosen folder's name, the “Sync now” button, and a short log of what the last sync did. Figure 14 shows the same panel when a sync reports a genuine conflict: the file is listed with its local/remote size and editor, alongside the Keep Local / Keep Remote / Keep Both actions.

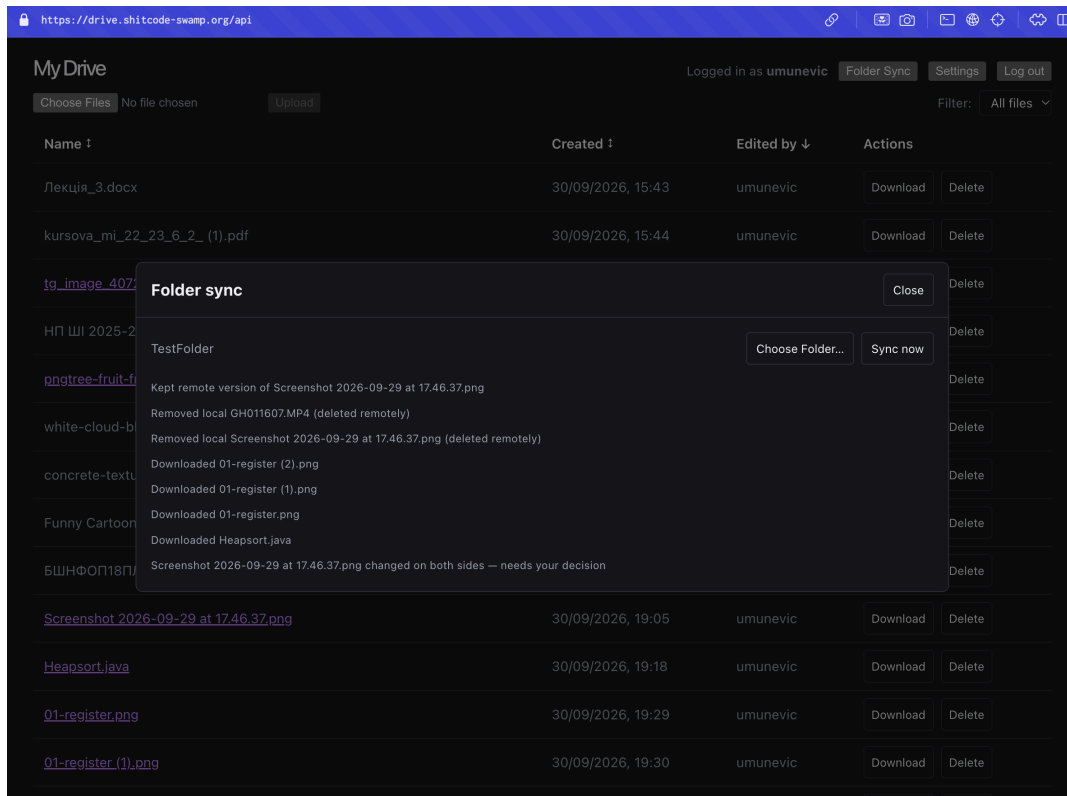


Figure 13: Folder sync panel after a completed sync

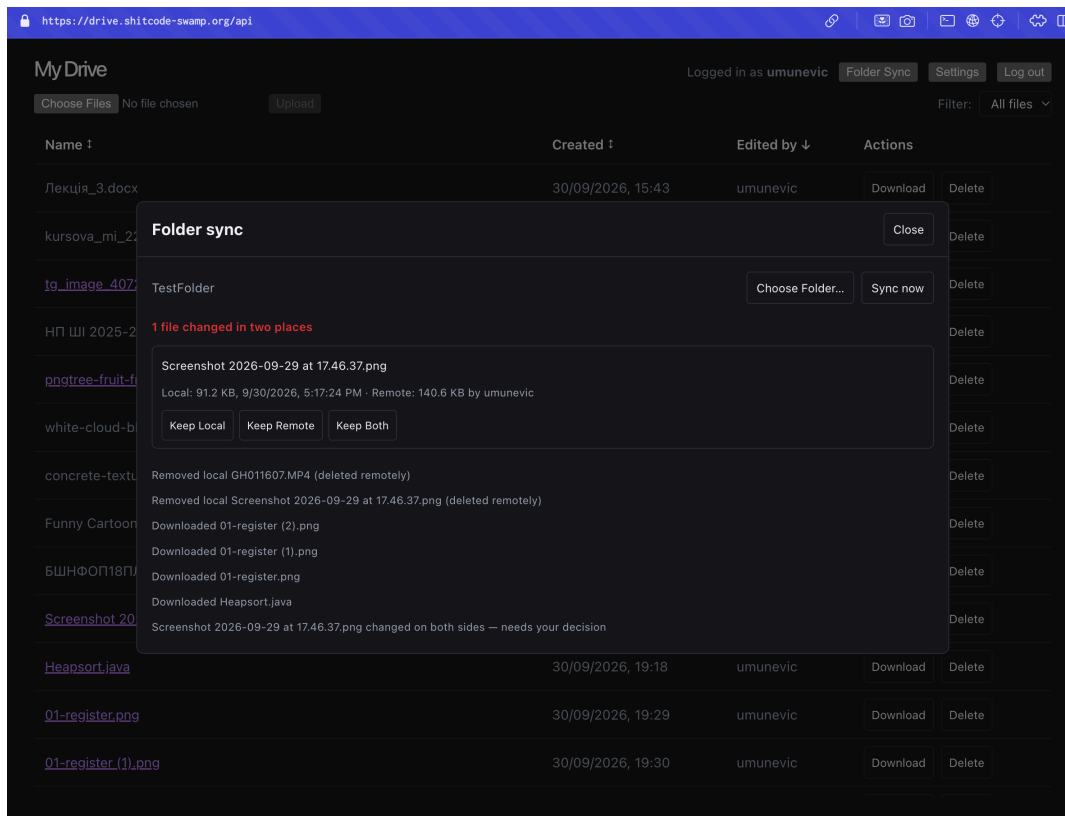


Figure 14: Folder sync panel showing a detected conflict and its resolution options

4. Unit Testing

4.1 Test Setup

The web client's tests run under Vitest with jsdom, using React Testing Library for component tests. Networked or browser-only collaborators are substituted so the suite runs deterministically and without a real backend, browser folder picker, or IndexedDB:

- `api/client.test.ts` stubs the global `fetch` and `XMLHttpRequest` (the latter used only by the chunked-upload path, since `fetch` has no upload-progress event) via `vi.stubGlobal`;
- `App.test.tsx` mocks only `listFiles/uploadFileInChunks` from the API client module, leaving every other export (token storage, the unauthorized-listener hook, ...) real, so `AuthContext` hydration behaves exactly as in production;
- `sync/reconcile.test.ts` and `sync/folderScanner.test.ts` use a small hand-rolled in-memory fake of `FileSystemDirectoryHandle/FileSystemFileHandle` (jsdom has no real File System Access API), and a fake API client mirroring the real module's exported functions;
- `sync/storage.ts` is written behind an injectable key-value store inter-

face specifically so `sync/storage.test.ts` can substitute an in-memory Map instead of real IndexedDB (also absent from jsdom);

- `sync/hash.test.ts` needs no mock at all – Node’s built-in `crypto.subtle` implements the real Web Crypto API, so the test hashes real bytes and checks them against the standard `sha256("")` vector.

4.2 Coverage by Area

Table 1 summarizes the test suite.

Table 1: Test suites in `frontend/src/`

Test file	Tests	What it covers
<code>api/client.test.ts</code>	11	Bearer-token attachment, the one-refresh-one-retry policy on a 401 (both the success and the refresh-also-fails path), response decoding including the X-Has-More header, and the chunked-upload path: splitting, per-chunk progress, abort-on-failure, 401-retry mid-upload, and the single-shot-vs-chunked size threshold
<code>auth/AuthContext.test.tsx</code>	5	Login/register/logout state transitions, <code>localStorage</code> persistence under the shared token keys, and hydrating an already-logged-in user from storage at mount without a fresh network call
<code>App.test.tsx</code>	3	The extension-filter dropdown’s option list staying independent of the currently-applied filter, and drag-and-drop upload including the drop overlay showing and hiding
<code>sync/hash.test.ts</code>	2	SHA-256 hex encoding, including the standard <code>sha256("")</code> test vector
<code>sync/folderScanner.test.ts</code>	3	Folder scanning: <code>size/mtime/sha256</code> computation, skipping dotfiles and subdirectories, and reporting an unreadable file separately from a missing one
<code>sync/storage.test.ts</code>	5	Sync state round-tripping through the key-value store, and <code>selectDirectory</code> ’s <code>reset-on-different-folder</code> versus <code>preserve-on-same-folder</code> behavior
<code>sync/reconcile.test.ts</code>	14	The reconcile algorithm’s <code>download/delete_local/conflict</code> handling, both same-cycle staleness-exclusion cases (a just-downloaded file, a just-server-deleted file), all three conflict resolutions, and the conflict-copy naming convention
Total	43	

Sorting and filtering themselves are applied server-side (the Go backend’s `sort/order/extension` query parameters, exercised by the backend’s own test suite); on the frontend side, `App.test.tsx` verifies that changing the filter sends the right parameter and that the dropdown’s own option list stays independent of it, rather than re-testing the backend’s sort/filter logic a second time.

Test Files 7 passed (7)
Tests 43 passed (43)

5. Conclusions

The web client covers the full set of core workspace use cases – browsing with server-driven sorting and filtering, content preview for `.java/.png`, upload (including drag-and-drop), download, delete – and, on Chromium-based browsers, folder synchronization against the same manifest-diff protocol and conflict model the desktop client uses, adapted to a manual one-shot action in place of a background watcher. Its architecture follows the same layering as the rest of the project: a thin `api/` client speaking the Go backend’s wire format, React Query/Table/Virtual managing server state and the file table, and the `sync/` folder isolating the File System Access API and IndexedDB behind the same kind of injectable seams the Swift client uses for its own testability. All 43 tests in the suite pass.